IoT Challenge 1 Report

Name: Rong Huang
Personal Code: 10948935

1 Introduction

This project aims to design a commercial-style IoT motion sensor node using an ESP32. The node detects human motion and measures ambient light intensity, then transmits this information to a sink node using the ESP-NOW protocol.

2 System Design

2.1 Hardware Setup

The system is implemented in the Wokwi simulator using:

- ESP32 microcontroller
- PIR motion sensor (digital input)
- LDR light sensor (analog input)

The PIR sensor is connected to GPIO 13, while the LDR analog output is connected to GPIO 34. The PIR module provides a digital signal indicating whether motion is detected, whereas the LDR module provides an analog signal representing the ambient light level.

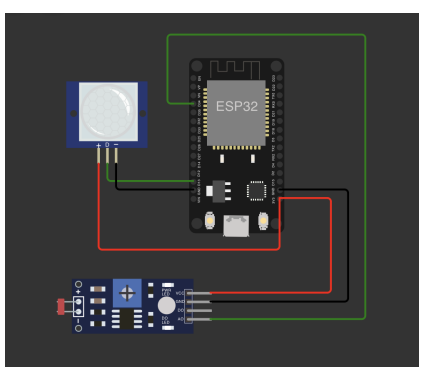

Figure 1: Wokwi hardware setup of the ESP32 sensor node.

2.2 Communication

The ESP-NOW protocol is used for communication between the sensor node and the sink node. In the Wokwi simulation, the broadcast address `FF:FF:FF:FF:FF:FF` is used.

The motion sensor output is read as a digital value, while the light sensor is read through the ESP32 ADC using `analogRead()`. The raw analog value is then converted in software into an approximate lux value, which is included in the transmitted message.

2.3 Message Format

The sensor node sends a string containing both motion and light information. Depending on the PIR reading, the transmitted message takes one of the following forms:

- `MOTION_DETECTED-LUMINOSITY:ZZZ`
- `MOTION_NOT_DETECTED-LUMINOSITY:ZZZ`

The message is built in the ESP32 code as follows:

```
String getMotionStatus() {
  return digitalRead(PIR_PIN) == HIGH ?
         "MOTION_DETECTED" : "MOTION_NOT_DETECTED";
}

message = motion + "-LUMINOSITY:" + String(luxInt);
```

2.4 Operating Cycle

Each operating cycle of the node consists of the following states:

1. BOOT
2. SENSOR READ
3. MESSAGE BUILD
4. WIFI ON
5. TRANSMISSION
6. WIFI OFF
7. DEEP SLEEP

In the implemented code, the states SENSOR READ, MESSAGE BUILD, WIFI ON, TRANSMISSION, and WIFI OFF are measured directly through `micros()`. The BOOT state is not taken from the simulator timing output, since the simulator does not realistically reproduce the actual hardware wake-up latency of a real ESP32 board. Instead, the BOOT duration is estimated later from the transient observed in `deep_sleep.csv`.

The deep sleep duration is determined from the personal code `10948935` according to the challenge rule:

$$X = \frac{(35 \bmod 50) + 5}{10} = 4.0 \text{ s}$$

Therefore, after each transmission cycle, the ESP32 enters deep sleep for 4 seconds.

2.5 State-to-Power Mapping from CSV Analysis

The provided CSV files were used as reference power traces to estimate the average power consumption of the operating states required by the implemented ESP32 node. These traces were not used as an exact one-to-one measurement of every line of code, but rather as representative power profiles for state-based energy estimation.

In particular, the three CSV files were interpreted as follows:

- `deep_sleep.csv`: used to identify the Deep Sleep, WiFi OFF, and WiFi ON plateaus
- `sensor-read.csv`: used as a reference for the generic sensor-reading phase and low-power local processing
- `sender.csv`: used to characterize transmission-related radio activity

Although the original dataset may include measurements collected under different radio conditions, in this work the traces are used only to extract representative average power values for the states required by the implemented node.

To estimate the state powers, the data were divided into stable regions according to the observed plateaus in the plots.

For `deep_sleep.csv`, three main regions were identified:

- low-power region (< 100 mW): Deep Sleep
- medium-power region ($200 - 300$ mW): WiFi OFF
- high-power region (> 500 mW): WiFi ON

For `sensor-read.csv`, two regions were identified:

- low-power region (< 260 mW): local processing state

- high-power region (> 330 mW): Sensor Read

For `sender.csv`, two relevant regions were identified:

- baseline region (< 620 mW): radio-active baseline
- spike region (> 640 mW): Transmission

State	Assigned Power (mW)
BOOT	237.05
SENSOR READ	351.09
MESSAGE BUILD	237.05
WIFI ON	632.04
TRANSMISSION	667.41
WIFI OFF	252.22
DEEP SLEEP	45.78

Table 1: Final state-to-power mapping obtained from the CSV analysis.

The mapping was performed using the custom Python script `group.py`. The output results were then saved in `csv_analysis_results.txt` for reference.

In particular, the BOOT power is approximated using the local low-power processing plateau, while the BOOT duration is derived later from the timestamped transient between the Deep Sleep plateau and the subsequent stable active plateau in `deep_sleep.csv`.

3 Experimental Measurements

3.1 Example Outputs

Two example serial outputs obtained from the Wokwi simulation are shown below. Since the BOOT state is estimated separately from the CSV analysis, the code output reports the active timing without BOOT.

```
MOTION_NOT_DETECTED-LUMINOSITY:21799
Sensor Read Time: 1.253 ms
Message Build Time: 0.318 ms
Transmission Time: 0.413 ms
WiFi ON Time: 190.372 ms
WiFi OFF Time: 9.619 ms
Total Active Time without BOOT: 201.975 ms
Cycle Period without BOOT: 4201.975 ms

MOTION_DETECTED-LUMINOSITY:21799
Sensor Read Time: 1.254 ms
Message Build Time: 0.359 ms
```

```
Transmission Time: 0.412 ms
WiFi ON Time: 190.330 ms
WiFi OFF Time: 9.614 ms
Total Active Time without BOOT: 201.969 ms
Cycle Period without BOOT: 4201.969 ms
```

3.2 Implementation Details

The ESP-NOW transmission is asynchronous. Therefore, simply calling `esp_now_send()` is not sufficient to guarantee that the packet has been fully transmitted before the node changes state.

To handle this correctly, a send callback function is registered. The code records the starting time immediately before calling `esp_now_send()`, and the ending time is recorded inside the callback when transmission completion is reported. In this way, the measured transmission time corresponds to the actual ESP-NOW sending interval rather than only the function call overhead.

The code directly measures the states SENSOR READ, MESSAGE BUILD, WIFI ON, TRANSMISSION, and WIFI OFF. Their sum is reported as the active time without BOOT. The BOOT state is not directly measured from the simulator output, but is estimated separately from the transient observed in `deep_sleep.csv`, since the simulator does not realistically represent the hardware wake-up latency of a real ESP32 board.

3.3 Measured Timing Results

The timing values reported in this section correspond only to the states that are directly measured in the ESP32 code, namely SENSOR READ, MESSAGE BUILD, TRANSMISSION, WIFI ON, and WIFI OFF. Therefore, the resulting active time is denoted as the active time without BOOT.

A total of 16 runs were collected from the simulator output. Two runs were removed as clear outliers because they exhibited abnormally large WiFi ON durations, which are not representative of the steady-state behavior of the node. Among the remaining 14 runs, Runs 1 and 2 were treated as warm-up runs, since their timing values were still noticeably higher and less stable than the following measurements. Therefore, only Runs 3–14 in Table 2 were used to compute the final average timing values.

Run	Sensor Read (ms)	Message Build (ms)	TX (ms)	WiFi ON (ms)	WiFi OFF (ms)	Active w/o BOOT (ms)
1	0.081	0.019	0.565	218.749	8.612	228.026
2	0.080	0.020	0.557	196.000	8.628	205.285
3	1.173	0.360	0.557	191.673	8.575	202.338
4	1.173	0.360	0.451	190.898	9.580	202.462
5	1.253	0.318	0.413	190.372	9.619	201.975
6	1.254	0.318	0.414	190.372	9.619	201.977
7	1.253	0.319	0.413	190.371	9.619	201.975
8	1.254	0.359	0.412	190.330	9.614	201.969
9	1.155	0.366	0.521	191.463	9.605	203.110
10	0.079	0.024	0.556	190.996	9.630	201.285
11	1.219	0.329	0.560	190.140	8.630	200.878
12	1.175	0.362	0.545	191.029	9.584	202.695
13	0.079	0.020	0.414	196.495	9.618	206.626
14	0.081	0.020	0.531	192.516	9.582	202.730
Average	0.929	0.263	0.482	191.388	9.440	202.502

Table 2: Measured timing results after removing the two clear outliers. The final average was computed from Runs 3–14, while Runs 1 and 2 were treated as warm-up runs.

State	Average Time (ms)
SENSOR READ	0.929
MESSAGE BUILD	0.263
TRANSMISSION	0.482
WIFI ON	191.388
WIFI OFF	9.440
ACTIVE without BOOT	202.502
Estimated BOOT	150.343
TOTAL ACTIVE	352.845
DEEP SLEEP	4000.000
CYCLE PERIOD	4352.845

Table 3: Final timing values used in the energy estimation.

Estimated BOOT. The code-measured active time reported in Table 3 does not include the BOOT state, because the simulator does not realistically reproduce the hardware wake-up latency of a real ESP32 board. Therefore, the BOOT duration was estimated separately from the timestamped power transient observed in `deep_sleep.csv`.

More precisely, for each cycle the BOOT interval was identified in the transition from the Deep Sleep low-power plateau to the subsequent stable medium-power plateau. The BOOT starting instant, denoted by $t_{\text{boot,start}}$, was defined as the first timestamp at which the signal departs from the Deep Sleep region. The BOOT ending instant, denoted by $t_{\text{boot,end}}$, was defined as the first

timestamp at which the signal reaches the stable medium-power plateau after the short transient spike. In the analyzed trace, the Deep Sleep plateau corresponds to the low-power region below 100 mW, while the stable post-boot region corresponds to the medium-power plateau around 200–300 mW.

Hence, the BOOT duration for each observed cycle was computed as:

$$T_{\text{boot}} = t_{\text{boot,end}} - t_{\text{boot,start}}$$

Three representative BOOT transients were extracted from `deep_sleep.csv`, giving:

$$T_{\text{boot},1} = 150.263 \text{ ms}, \qquad T_{\text{boot},2} = 150.468 \text{ ms}, \qquad T_{\text{boot},3} = 150.299 \text{ ms}$$

The average estimated BOOT time was then computed as:

$$T_{\text{boot,avg}} = \frac{T_{\text{boot},1} + T_{\text{boot},2} + T_{\text{boot},3}}{3} = 150.343 \text{ ms}$$

Therefore, the final total active time used in the energy estimation is obtained by:

$$T_{\text{active}} = T_{\text{active,without boot}} + T_{\text{boot,avg}}$$

Using the values reported in Table 3,

$$T_{\text{active}} = 202.502 + 150.343 = 352.845 \text{ ms}$$

and the corresponding cycle period is:

$$T_{\text{cycle}} = T_{\text{active}} + T_{\text{sleep}} = 352.845 + 4000 = 4352.845 \text{ ms}$$

4 Energy Consumption Model and Estimation

4.1 Power and Energy Model

The energy consumed in one operating cycle is computed as the sum of the energy contributions of all states:

$$E_{\text{cycle}} = E_{\text{boot}} + E_{\text{sensor}} + E_{\text{msg}} + E_{\text{wifi_on}} + E_{\text{tx}} + E_{\text{wifi_off}} + E_{\text{sleep}}$$

For each state, the corresponding energy is computed as:

$$E_i = P_i \cdot T_i$$

where P_i is the average power of the i-th state and T_i is the corresponding duration.

Expanding the expression for one full cycle:

$$E_{\text{cycle}} = P_{\text{boot}}T_{\text{boot}} + P_{\text{sensor}}T_{\text{sensor}} + P_{\text{msg}}T_{\text{msg}} + P_{\text{wifi_on}}T_{\text{wifi_on}} + P_{\text{tx}}T_{\text{tx}} + P_{\text{wifi_off}}T_{\text{wifi_off}} + P_{\text{sleep}}T_{\text{sleep}}$$

In this work, power is expressed in mW and time in ms, therefore the energy is reported in mJ according to:

$$E_i(\text{mJ}) = \frac{P_i(\text{mW}) \cdot T_i(\text{ms})}{1000}$$

4.2 Input Values for Energy Estimation

The timing values used in the energy estimation are taken from the experimental results reported in Section 3, while the power values are obtained from the CSV-based state-to-power mapping reported in Section 2.

In particular, the final timing values are summarized in Table 3, and the assigned power values are reported in Table 1. The BOOT duration is not measured directly from the simulator output, but is estimated from the transient observed in `deep_sleep.csv`, as discussed in Section 3.

4.3 Energy Estimation per Cycle

Using the average timing values and the extracted power values, the energy consumption of each state is computed as follows:

State	Time (ms)	Power (mW)	Energy (mJ)
BOOT	150.343	237.05	35.6388
SENSOR READ	0.929	351.09	0.3262
MESSAGE BUILD	0.263	237.05	0.0623
WIFI ON	191.388	632.04	120.9642
TRANSMISSION	0.482	667.41	0.3217
WIFI OFF	9.440	252.22	2.3809
DEEP SLEEP	4000.000	45.78	183.1200
TOTAL	–	–	342.8141

Table 4: Energy consumption per state in one operating cycle.

Therefore, the total energy consumed in one operating cycle is:

$$E_{\text{cycle}} \approx 342.81 \text{ mJ}$$

4.4 Average Power and Battery Lifetime

The average power over one operating cycle is computed as:

$$P_{\text{avg}} = \frac{E_{\text{cycle}}}{T_{\text{cycle}}}$$

Using:

$$E_{\text{cycle}} = 0.3428141 \text{ J} \qquad T_{\text{cycle}} = 4.352845 \text{ s}$$

we obtain:

$$P_{\text{avg}} = \frac{0.3428141}{4.352845} \approx 0.07875 \text{ W}$$

that is,

$$P_{\text{avg}} \approx 78.75 \text{ mW}$$

Given the battery energy specified by the challenge rule and the personal code `10948935`, the last four digits are 8935. Therefore,

$$E_{\text{battery}} = 8935 \bmod 5000 + 15000 = 3935 + 15000 = 18935 \text{ J}$$

The expected device lifetime is:

$$L = \frac{E_{\text{battery}}}{P_{\text{avg}}}$$

Hence:

$$L = \frac{18935}{0.07875} \approx 240434.72 \text{ s}$$

which corresponds to:

$$L \approx 66.79 \text{ hours} \approx 2.78 \text{ days}$$

The number of operating cycles supported by the battery is:

$$N_{\text{cycles}} = \frac{E_{\text{battery}}}{E_{\text{cycle}}} = \frac{18935}{0.3428141} \approx 55234$$

5 Improvements

The baseline implementation described in Sections 2–4 transmits a message at every operating cycle. As shown in Table 4, the **WIFI ON** state is the dominant high-power phase of the cycle ($P_{\text{wifi_on}} = 632.04\,\text{mW}$). Consequently, transmitting at every cycle can lead to unnecessary radio activations and higher average energy consumption when the sensed conditions remain unchanged.

To reduce energy consumption without changing the main objective of the sensor node (reporting motion and ambient light conditions to a sink node), a **send-on-change** strategy is adopted and supported by an updated energy/power analysis.

5.1 Send-on-change policy and light quantization

Transmission is performed only when (i) the PIR motion state changes, or (ii) the ambient light category changes. To avoid transmissions caused by small lux fluctuations (ADC noise or minor variations), the light measurement (0–100000 lux) is quantized into five discrete levels (L0–L4). As a result, only category transitions trigger transmissions, while minor changes within the same category are ignored.

5.2 Evidence from reimplementation on Wokwi

Implementation note (offline simulator). In offline Wokwi-local, ESP32 deep sleep and RTC persistence are not reliably emulated; repeated deep sleep cycles may appear as power-on resets. For this reason, the send-on-change logic is validated using a periodic loop with a fixed operating period $T = 8\,\mathrm{s}$, while keeping the same per-state power model from Table 1. For energy recomputation, the BOOT duration is kept equal to the baseline estimate ($t_{\mathrm{BOOT}} = 150.343\,\mathrm{ms}$) to approximate a wake-up cycle.

After applying the optimization, two cycle types are observed in the serial output:

- **SEND cycle** (`needSend=YES`): ESP-NOW is activated and a packet is transmitted.
- **SKIP cycle** (`needSend=NO`): no transmission is performed and WiFi remains OFF.

In a 16-cycle run, transmission occurs in $N_{\mathrm{SEND}} = 7$ cycles and is skipped in $N_{\mathrm{SKIP}} = 9$ cycles. Compared to the baseline (always transmitting), the radio activation frequency is reduced by

$$\frac{9}{16} \approx 56.25\%.$$

Example serial outputs. The following excerpts illustrate the two observed cycle types (SKIP vs SEND) after applying the send-on-change policy:

```
========== Run 16 ==========

MotionNow: 0 | Lux: 1310.26 | LevelNow: 2 (L2_INDOOR)
LastMotion: 0 | LastLevel: 2
needSend: YES | motionChanged: NO | levelChanged: YES
Sensor Read Time: 919 us
Message Build Time: 495 us
WiFi ON Time: 29.802 ms
Transmission Time: 443 us
WiFi OFF Time: 8434 us
Sent: MOTION_NOT_DETECTED-LEVEL:2-LUX:1310
```

```
Total Active Time (sim cycle, no deep sleep): 40.093 ms
Cycle Period (sim): 8040.093 ms
```

Serial logs. The complete 16-cycle serial output is provided in the submission package under `result/serial_output_improved.txt`.

5.3 Energy estimation after improvements

To recompute the energy profile after the improvements, representative timing values are extracted from the serial logs: one SEND cycle (e.g., Run 8) and one SKIP cycle (e.g., Run 6). The sleep/idle time is $t_{\text{SLEEP}} = 8000\,\text{ms}$. The power values are unchanged from Table 1. Energy is computed as in Section 4:

$$E_i(\text{mJ}) = \frac{P_i(\text{mW}) \cdot T_i(\text{ms})}{1000}.$$

State	Time (ms)	Power (mW)	Energy (mJ)
BOOT	150.343	237.05	35.6388
SENSOR READ	0.930	351.09	0.3265
MESSAGE BUILD	0.498	237.05	0.1181
WIFI ON	29.790	632.04	18.8302
TRANSMISSION	0.567	667.41	0.3785
WIFI OFF	8.618	252.22	2.1736
SLEEP/IDLE	8000.000	45.78	366.2400
TOTAL	–	–	423.7057

Table 5: Energy consumption per state in a representative **SEND** cycle after improvements ($T = 8\,\text{s}$).

State	Time (ms)	Power (mW)	Energy (mJ)
BOOT	150.343	237.05	35.6388
SENSOR READ	0.911	351.09	0.3198
SLEEP/IDLE	8000.000	45.78	366.2400
TOTAL	–	–	402.1986

Table 6: Energy consumption per state in a representative **SKIP** cycle after improvements ($T = 8\,\text{s}$).

Average energy and average power. Let E_{SEND} and E_{SKIP} denote the energies of a SEND and SKIP cycle from Tables 5–6. Over 16 cycles, the total energy is:

$$E_{\text{tot}} = 7E_{\text{SEND}} + 9E_{\text{SKIP}} \approx 7 \cdot 423.7057 + 9 \cdot 402.1986 \approx 6585.72\,\text{mJ}.$$

With a fixed operating period $T = 8\,\mathrm{s}$, the total time is $T_{\mathrm{tot}} = 16 \cdot 8 = 128\,\mathrm{s}$, thus the average power after improvements is:

$$\bar{P}_{\mathrm{after}} = \frac{E_{\mathrm{tot}}}{T_{\mathrm{tot}}} = \frac{6.58572\,\mathrm{J}}{128\,\mathrm{s}} \approx 0.05145\,\mathrm{W} \approx 51.45\,\mathrm{mW}.$$

Comparison with baseline and practical impact. From Section 4, the baseline average power is $P_{\mathrm{avg}} \approx 78.75\,\mathrm{mW}$. After improvements, the average power is reduced to $\bar{P}_{\mathrm{after}} \approx 51.45\,\mathrm{mW}$, corresponding to an approximate reduction of:

$$1 - \frac{\bar{P}_{\mathrm{after}}}{P_{\mathrm{avg}}} \approx 34.7\%.$$

The reduction is mainly due to the elimination of WIFI ON / TRANSMISSION / WIFI OFF phases in SKIP cycles, as indicated by the `SKIP TX` events in the serial logs. In practical terms, fewer radio activations reduce the average energy consumption while preserving meaningful updates to the sink node.

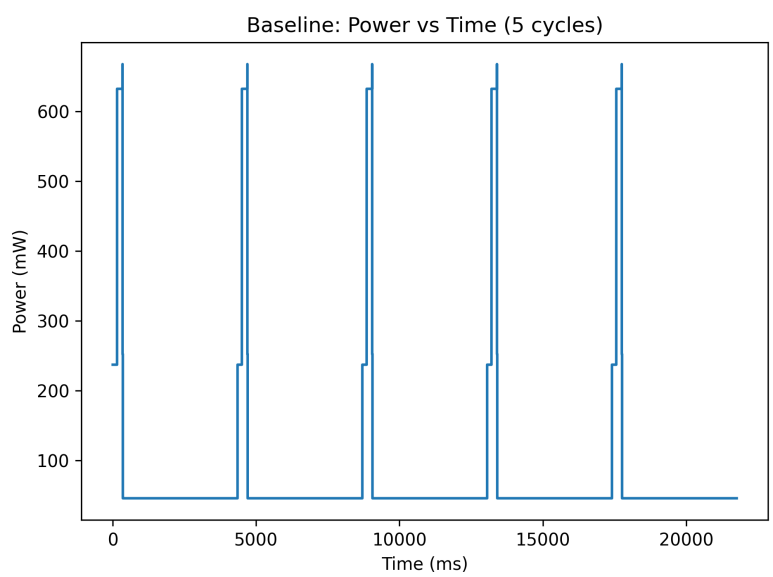

Figure 2: Baseline power vs time over 5 operating cycles. High-power spikes (WIFI ON and TRANSMISSION) occur in every cycle.

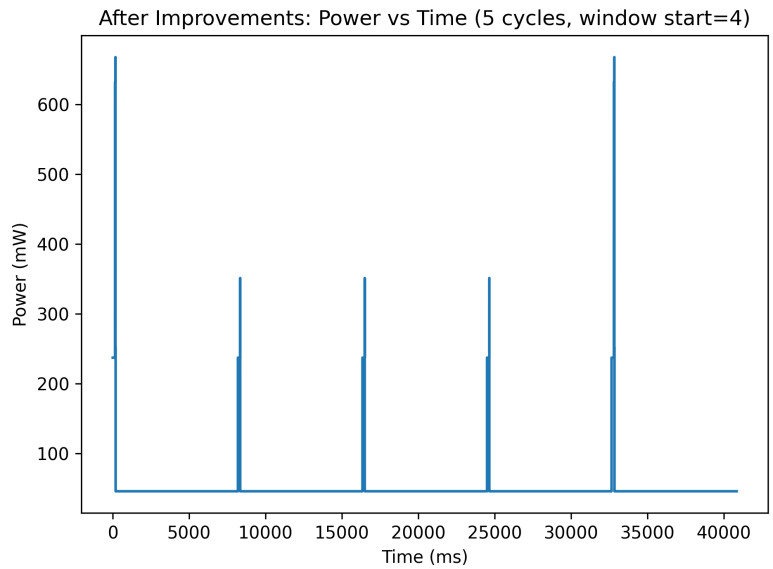

Figure 3: Power vs time after improvements over 5 cycles. Several cycles do not activate the radio (SKIP cycles), thus high-power spikes are reduced.

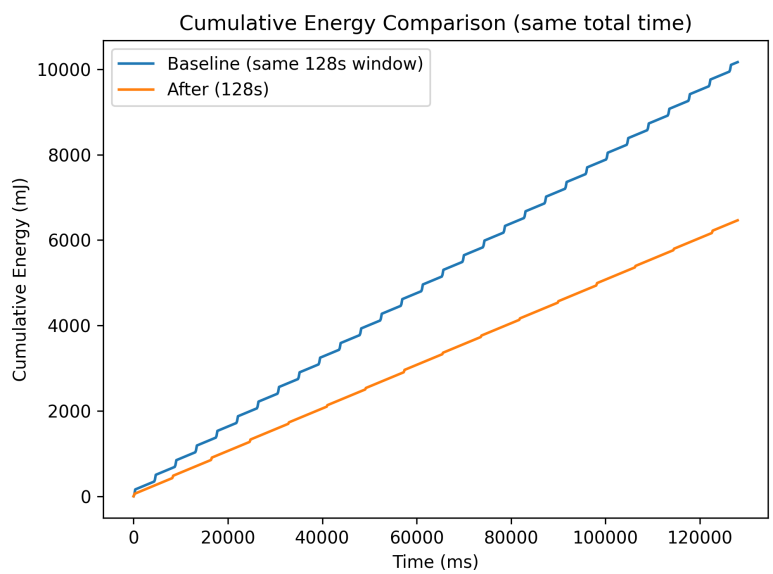

Figure 4: Cumulative energy comparison over the same total time window (128 s). The improved design shows a lower energy growth rate, indicating reduced average power consumption.

Serial logs. The complete 16-cycle serial output for the improved implementations is included in the submission package (`result/serial_output_improved.txt`) to support reproducibility.

Equivalent 4-second cycle energy. Using the post-improvement average power $\bar{P}_{\text{after}} \approx 51.45\,\text{mW}$, the equivalent energy for a 4 s cycle is $E_{\text{after},4s} = \bar{P}_{\text{after}} \cdot 4\,\text{s} \approx 205.8\,\text{mJ}$.

6 Energy Trade-off in a WSN

Consider the linear topology shown in the assignment, where a motion sensor must deliver one packet to the sink node either by direct transmission or by multi-hop forwarding through three relay nodes (B1, B2, B3). The total distance is $D = 40\,\text{m}$, and nodes are equally spaced. Hence, in the multi-hop case the hop distance is:

$$d = \frac{D}{4} = \frac{40}{4} = 10\,\text{m}.$$

Each message has size $k = 2000$ bits and the communication energy model is:

$$E_{\text{tx}}(k, d) = kE_c + k\varepsilon d^2, \qquad E_{\text{rx}}(k) = kE_c,$$

with $E_c = 300\,\text{nJ/bit}$ and $\varepsilon = 1\,\text{nJ/(bit} \cdot \text{m}^2)$. Ideal communication is assumed (no retransmissions and no idle energy).

6.1 Q1: Total communication energy for one message

(a) Direct transmission. In the direct case, the sensor transmits once over $d = D = 40\,\text{m}$ and the sink receives once. First, compute the common term:

$$kE_c = 2000 \cdot 300 = 600000 \text{ nJ}.$$

The distance-dependent transmit term is:

$$k\varepsilon d^2 = 2000 \cdot 1 \cdot 40^2 = 2000 \cdot 1600 = 3200000 \text{ nJ}.$$

Therefore:

$$E_{\text{tx}}(2000, 40) = 600000 + 3200000 = 3800000 \text{ nJ}, \qquad E_{\text{rx}}(2000) = 600000 \text{ nJ}.$$

The total network communication energy for one delivered message is:

$$E_{\text{direct}} = E_{\text{tx}}(2000, 40) + E_{\text{rx}}(2000) = 3800000 + 600000 = 4400000 \text{ nJ} = 4.4 \text{ mJ}.$$

(b) Multi-hop transmission. In the multi-hop case, the message is forwarded over 4 hops of $d = 10\,\mathrm{m}$. For one hop:

$$k\varepsilon d^2 = 2000 \cdot 1 \cdot 10^2 = 2000 \cdot 100 = 200000 \text{ nJ},$$

thus:

$$E_{\mathrm{tx}}(2000, 10) = kE_c + k\varepsilon d^2 = 600000 + 200000 = 800000 \text{ nJ}.$$

Across the whole path, there are 4 transmissions (sensor, B1, B2, B3) and 4 receptions (B1, B2, B3, sink). Hence:

$$E_{\mathrm{multi}} = 4{\cdot}E_{\mathrm{tx}}(2000, 10){+}4{\cdot}E_{\mathrm{rx}}(2000) = 4{\cdot}800000{+}4{\cdot}600000 = 3200000{+}2400000 = 5600000\,\mathrm{nJ} = 5.6\,\mathrm{mJ}.$$

6.2 Q2: Strategy for long-term operation and optimized metric

The choice depends on the performance metric.

Metric: total network energy per delivered message. Direct transmission is preferable because it minimizes the total communication energy:

$$E_{\mathrm{direct}} = 4.4\,\mathrm{mJ} \;<\; E_{\mathrm{multi}} = 5.6\,\mathrm{mJ}.$$

Metric: network lifetime (bottleneck node energy). Multi-hop transmission reduces the transmission distance of the motion sensor from 40 m to 10 m, thereby reducing the sensor's transmit cost per message and distributing energy consumption across relay nodes. This can be preferable when the motion sensor is the energy-constrained (battery-powered) device and relays are less constrained. The trade-off is that intermediate nodes (B1–B3) must both receive and transmit, increasing their communication energy compared to the direct case.

In summary, direct transmission minimizes total network communication energy per message, while multi-hop can be preferable when the objective is to extend the lifetime of the most energy-constrained node by reducing its long-distance transmissions.

Final choice. For long-term operation, the optimized metric is the lifetime of the energy-constrained motion sensor. Therefore, multi-hop forwarding is selected because it reduces the sensor transmission distance from 40 m to 10 m, at the cost of additional relay receptions/transmissions, consistent with the numerical results in Q1.

7 Conclusion

This report implemented an ESP32-based motion and light sensor node using ESP-NOW and measured the timing of the main operating states. Using a state-based power mapping extracted from the provided CSV traces, the baseline design consumes approximately $E_{\text{cycle}} \approx 342.81\,\text{mJ}$ per cycle, corresponding to an average power of about $78.75\,\text{mW}$ and an estimated lifetime of ≈ 2.78 days under the given battery model.

To reduce unnecessary radio activations, a send-on-change strategy with light-level quantization was introduced. The improved design reduces the radio activation frequency (9 out of 16 cycles skipped transmission) and lowers the average power to approximately $51.45\,\text{mW}$, i.e., about 34.7% lower than the baseline, as supported by the power/energy plots.

Finally, the WSN trade-off analysis showed that direct transmission minimizes total network energy per message, while multi-hop forwarding can be preferable when the objective is to extend the lifetime of the energy-constrained sensor node.